

DISCORD-RATISS — L'agent qui parle au serveur du labo

Deep Dive n°12 de la série « Les découvertes du laboratoire RATISS » (20-28 septembre 2026)

DOCUMENT SOURCE OFFICIEL — LABORATOIRE RATISS LABS Auteur et fondateur : **Jonathan Evina**, Cameroun. Dépôt : github.com/jonathansearch/DISCORD-RATISS · MIT. Rédigé depuis les sources primaires : `agent.py` (code de l'agent), `workflows hub-central.yml` et `notifier.yml`, JOURNAL des opérations (runs, HTTP 204, tests de purge). Aucun secret ni URL de webhook n'apparaît dans ce document — c'est une règle absolue du labo.

1. L'histoire en trois minutes

Un labo qui produit des résultats à cette vitesse a besoin d'un **héraldiste** : quelque chose qui annonce les preuves, les échecs et les campagnes dans son serveur Discord — sans que le chef ait à copier-coller quoi que ce soit.

DISCORD-RATISS est cet agent. Un script Python volontairement **autonome** (`agent.py` : un seul fichier, aucune dépendance externe) appelé par des workflows GitHub Actions, qui poste dans les salons du labo via des webhooks. Ses principes, gravés dans le code :

1. **Le secret n'est jamais exposé** : il arrive par variable d'environnement, jamais en argument, jamais écrit sur disque, jamais affiché — les logs masquent tout.
2. **Aucune mention possible** : `allowed_mentions.parse = []` — l'agent ne peut techniquement pinger personne.
3. **Continuation si secret absent** : un salon non configuré ne casse jamais la chaîne des autres.
4. **Tout est vérifié** : chaque envoi est journalisé (HTTP 204), chaque run est inspecté après coup, les messages supprimés sont comptés.

Les jalons de la semaine : vérification d'empreintes **55/55 conformes** postée le 26/09 à 15:21 UTC ; hub multi-salons routé sur **23 secrets individuels** (RATISS → RATISS23) ; **test global du 27/09 : 22 salons répondent HTTP 204** en un seul run ; les **annonces PHOTON et ETALONS** publiées le 27/09 à 22:05 UTC ; et un **mode purge** ajouté le 28/09 — l'agent sait désormais supprimer ses propres messages (jamais ceux du chef).

2. Carte d'identité du dépôt

Élément	Valeur exacte
Nom	DISCORD-RATISS — hub de notification RATISS LABS
Institution	RATISS Labs (Cameroun) — projet indépendant mono-auteur
Création	26 septembre 2026
Composants	agent.py (agent autonome) · hub-central.yml (hub multi-salons) · notifier.yml (annonces + rapport quotidien 08:00 UTC)
Couverture	23 secrets de salons (RATISS → RATISS23) + mode <code>all</code> — 22 salons actifs
Sécurité	secrets env-only, logs masqués, <code>allowed_mentions.parse = []</code> , continuation si absent
Jalons	55/55 empreintes (26/09) · 22× HTTP 204 (27/09) · purge testée (28/09)
Licence	MIT

3. L'agent : un seul fichier, quatre pouvoirs

`agent.py` est écrit avec une règle d'or : **un point d'entrée de CI ne dépend d'aucun fichier voisin**. Ses quatre usages :

```
python3 agent.py # RAPPORT : vérifie RATISS-ARCHIVES et poste le verdict
python3 agent.py --test # message de connexion
python3 agent.py --titre "..." --details "..." --statut OK --lien "..."
python3 agent.py --salon general --titre "..." # choisit le webhook du salon
python3 agent.py --purger # supprime les messages postés par l'agent
```

Le **rapport** est le vrai travail quotidien : cloner RATISS-ARCHIVES, recalculer toutes les empreintes SHA-256 du MANIFESTE, et poster le verdict (✓ « 55/55 empreintes conformes » ou ✗ avec le détail). Les messages sont des **embeds** colorés (vert OK, rouge ÉCHEC, bleu INFO) signés « RATISS LABS · agent du hub ».

La **purge** (ajoutée le 28/09 sur ordre du chef) utilise la propriété Discord qui interdit à un webhook de supprimer que **ses propres** messages : l'agent balaie ses 23 secrets, lit son dernier message (`@original`) et le supprime, jusqu'à vide — avec un garde-fou de 10 suppressions par salon et par run. Testé contre un faux serveur Discord en local (1 message, 3 messages, salon déjà propre, secret absent) avant mise en production, puis validé en réel : **run success, 0 message restant** (le chef avait déjà nettoyé à la main).

4. Le hub multi-salons : 22 salons, un seul run

Le workflow `hub-central.yml` est le **poste de commandement**. Une entrée `salon` (un nom logique, ou `all`), une entrée `statut`, `titre`, `details`, `lien`, plus `test_seul` (sonde de connexion) et `purger` (nettoyage).

Le routage repose sur **23 secrets GitHub** nommés RATISS, RATISS2, ... RATISS23 — un par salon. Le mode `all` envoie à tous les secrets **non vides**, en **continuant** si l'un est absent. L'étape de vérification préalable valide que chaque webhook est bien une URL Discord — **sans jamais l'afficher**.

Le grand test du 27/09 (run 36357382700) : `{"salon": "all", "test_seul": true}` → **22 envois HTTP 204** (le 23e secret, RATISS11, était vide — identifié proprement par l'absence d'empreinte dans les logs masqués, et signalé au chef comme action à faire). Les logs d'archive ont été comptés automatiquement : 22 « ✓ message envoyé (HTTP 204) » uniques.

Et les premières **vraies missions** : le 27/09 à 22:05 UTC, les annonces PHOTON et ETALONS sont publiées dans les salons via `notifier.yml` — deux runs success, deux fois HTTP 204.

5. Les règles de sécurité (celles qui ne se négocient pas)

Le labo les a établies dès le premier jour, et tout le code les reflète :

- Ne **jamais** afficher la valeur d'un secret, ni l'URL d'un webhook — pas dans les logs, pas dans un fichier, pas dans une conversation.
- Le secret arrive **uniquement** par variable d'environnement injectée par GitHub.
- `allowed_mentions.parse = []` sur chaque envoi : aucune mention, même accidentelle.
- Si un secret est absent : on **saute** le salon et on continue — jamais de plantage qui fuirait l'état de la configuration.
- Les erreurs Discord sont signalées avec leur code HTTP, sans jamais révéler l'URL fautive.
- Les logs sont relus après chaque run avec un garde-fou automatique (aucune vraie URL ne doit apparaître).

6. La chaîne complète : du commit au salon

Le flux typique d'une découverte RATISS :

1. Le labo publie un résultat (ex. PHOTON : fidélité 95,9–96,8 %, sceau 33/33).
2. L'agent reçoit l'ordre (dispatch `hub-central.yml` ou `notifier.yml`).
3. `agent.py` construit l'embed (titre, détails, couleur selon le statut, lien vers le dépôt).
4. Le hub route vers le ou les salons (secrets non vides uniquement).
5. Discord répond 204 par salon ; le run est vérifié après coup (logs comptés).
6. Le chef lit l'annonce dans son serveur — sans avoir rien fait lui-même.

Et en parallèle, **chaque jour à 08:00 UTC**, le rapport d'empreintes se poste tout seul : la mémoire d'ARCHIVES surveillée en continu.

7. Ce que le dépôt n'est pas

- Ce n'est **pas** un bot Discord complet (pas de WebSocket, pas de commandes interactives) : c'est un **héraldiste** unidirectionnel, volontairement minimal.

- Il ne lit **pas** les messages du serveur — il ne parle que.
 - Il ne garantit pas la disponibilité (dépend de GitHub Actions et des webhooks), mais il **journalise** chaque échec de façon non-fuyante.
 - La purge ne touche que **ses propres** messages — propriété de la plateforme Discord, pas une permission accordée.
-

8. Les phrases à retenir

- « Le secret arrive par variable d'environnement — jamais en argument, jamais écrit sur disque, jamais affiché. » (en-tête du code)
 - « 22x ✓ message envoyé (HTTP 204). » (le test global du 27/09)
 - « Un webhook ne peut supprimer que ses propres messages : ceux du chef ne sont jamais touchés. » (la purge)
 - « Pour chaque résultat : annoncé, vérifié, compté. » (la méthode des runs)
-

9. FAQ du Deep Dive

Q : Pourquoi un dépôt juste pour poster des messages ? R : Parce que l'annonce **vérifiée** fait partie de la méthode : un résultat RATISS est publié, scellé, **et annoncé** avec ses chiffres et son lien. L'agent transforme cette étape en routine automatique — 22 salons en un run — sans jamais exposer la moindre clé.

Q : Comment l'agent sait-il où poster sans montrer les URLs ? R : Chaque salon est un **secret GitHub** nommé RATISS → RATISS23. Le code ne voit que des variables d'environnement, valide leur forme (préfixe Discord) sans les afficher, et les logs sont masqués par la plateforme — vérifié par un garde-fou qui scanne les logs après coup.

Q : RATISS11, c'est quoi l'histoire ? R : Un des 23 secrets était **vide**. L'architecture de continuation a permis de l'identifier proprement (le salon absent de la liste plutôt qu'un crash), et le test global a tourné : 22/22 salons configurés répondent. RATISS11 reste à remplir par le chef — la mémoire du labo le note comme action ouverte.

Q : La purge, elle peut effacer des messages du chef ? R : Non — et c'est **Discord** qui l'interdit, pas juste le code : un webhook ne peut supprimer que ses propres messages. Le mode a été simulé en local (faux serveur Discord, 4 cas de test) puis validé en réel (run success).

Q : Qui est derrière ? R : RATISS Labs, laboratoire indépendant mono-auteur de Jonathan Evina (Cameroun). MIT, tout public — hors secrets, évidemment.

Q : Comment vérifier sans rien casser ? R : Le workflow a un mode `test_seul` (sonde de connexion) et un mode `dry-run` (affiche le JSON sans envoyer). Et chaque run est inspectable publiquement dans l'onglet Actions.

10. Fiche de fin

RATISS Labs · Jonathan Evina · Cameroun · Septembre 2026 · MIT. Document Deep Dive n°12/14 — série « Les découvertes RATISS du 20 septembre » — généré depuis les sources primaires du dépôt.